

MACHINE LEARNING OPERATIONS

Production-Ready Heart Disease Risk Classification

End-to-end model development, CI/CD, deployment and monitoring

Module: AIMLCZG523 — Machine Learning Operations

Assessment: Assignment 01 · Total marks: 50

Student: Amruthavalli Kuncha

Student ID: 2024ac05 402

Submission: [insert submission date]

Educational system — not for clinical diagnosis

Executive summary

This project turns the official UCI Cleveland heart-disease cohort into a reproducible binary classifier and an observable API. Two model families were tuned with five-fold stratified cross-validation. Logistic regression was selected by the declared criterion—highest training-partition CV ROC-AUC (0.911)—and reached 0.932 ROC-AUC on the untouched test set.

The implementation is script-first and packages preprocessing with the estimator. MLflow records experiments; FastAPI serves predictions; Pytest and Ruff gate quality; Docker and Kubernetes package deployment; Prometheus and Grafana expose operational signals.

System architecture

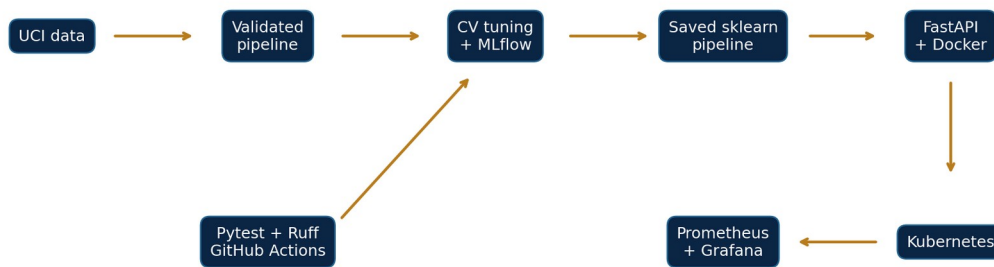


Figure 1. Reproducible path from data acquisition to monitored deployment.

Verified outcome

- 303 official UCI records; 13 predictors; 45.87% positive outcome.
- Six tests passing with 92% line coverage; lint checks passing.
- Selected model: logistic regression, C=0.2, no class weighting.
- Saved artifact includes imputation, encoding, scaling and classifier.

1. Data acquisition and exploratory analysis

The processed Cleveland cohort is downloaded directly from the UCI Machine Learning Repository. The raw file digest is a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8, providing an auditable data version. Original severity values 1–4 are mapped to presence and 0 to absence.

The cleaning audit found no duplicate records, four missing values in ca and two in thal. Missing predictors are retained for leakage-safe pipeline imputation; the outcome has no missing values.

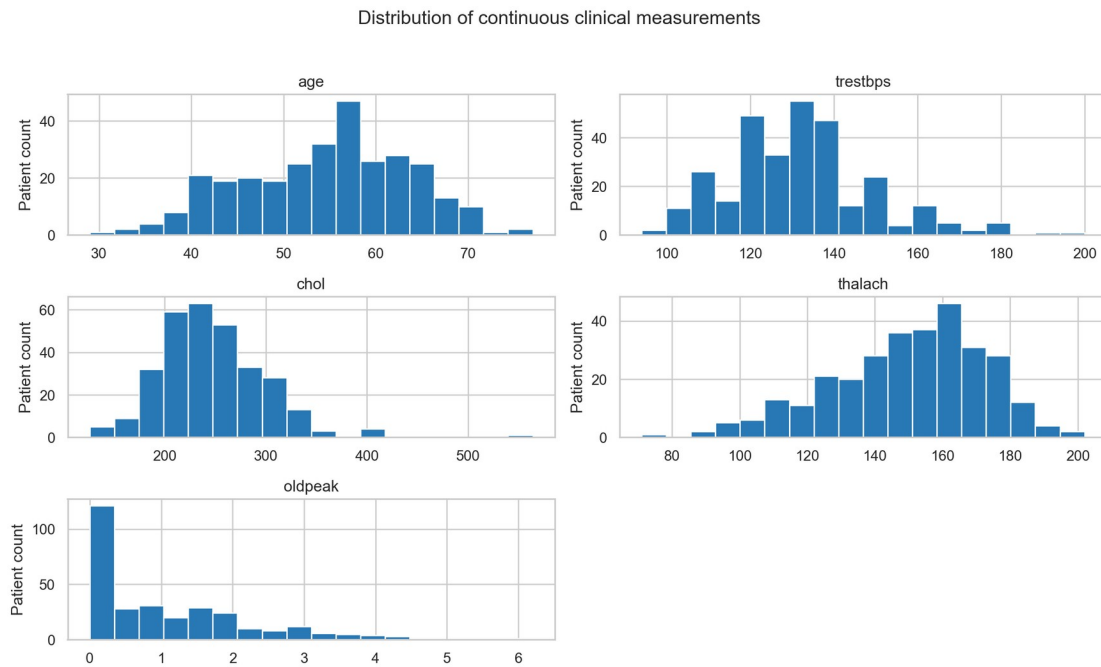


Figure 2. Continuous-feature distributions across 303 records.

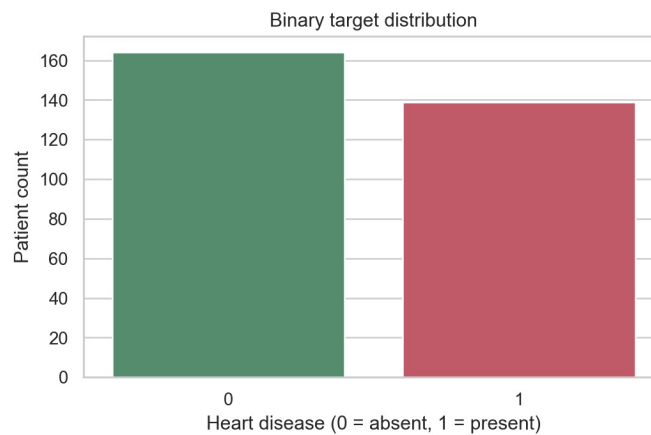


Figure 3. Binary outcome distribution (positive rate 45.87%).

2. Feature engineering and evaluation design

Numeric features—age, resting blood pressure, cholesterol, maximum heart rate and ST depression—use median imputation with missingness indicators and standardization. Eight coded features use most-frequent imputation and one-hot encoding with unknown-category tolerance.

An 80/20 stratified split with seed 523 produces 242 training and 61 held-out records. Grid search operates only on the training partition using five shuffled stratified folds and ROC-AUC scoring. This keeps test data outside preprocessing and selection.

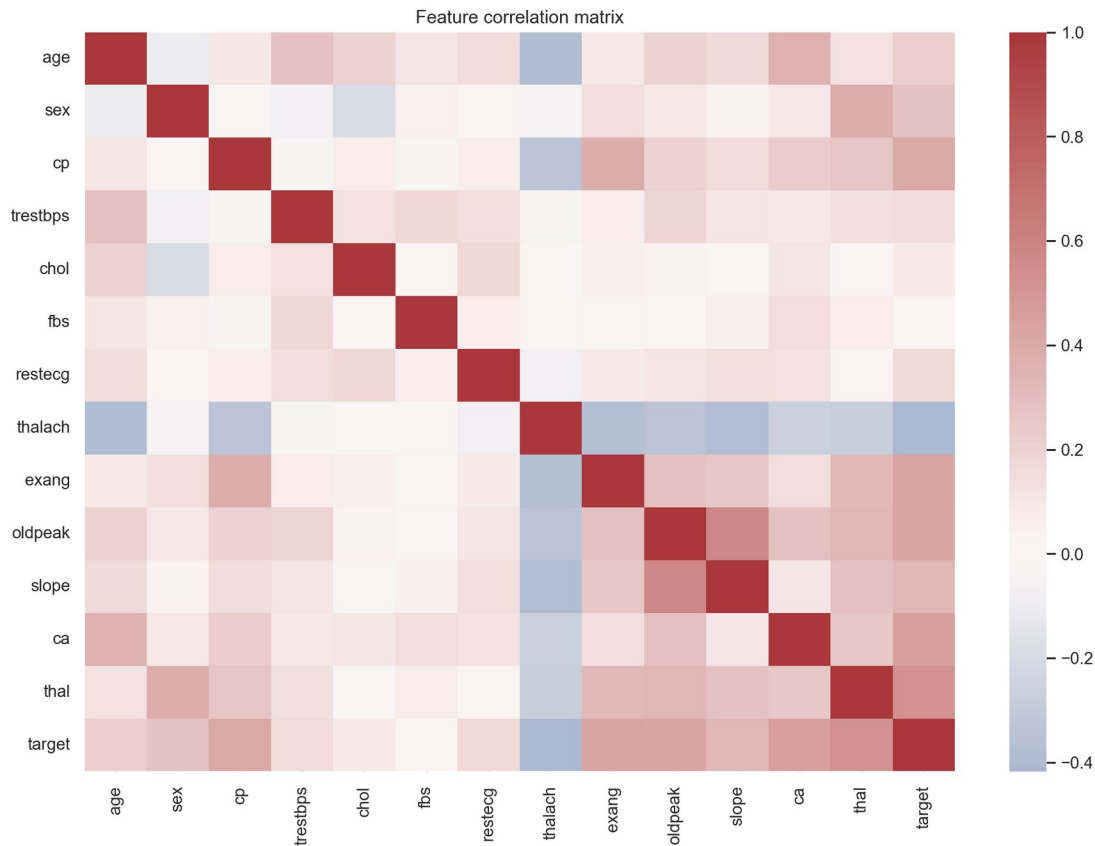


Figure 4. Numeric correlation matrix; associations are descriptive, not causal.

3. Model development and selection

Logistic regression explores four regularization strengths and optional balanced weights. Random forest explores tree count, depth, minimum leaf size and optional balanced weights. The selection rule is highest cross-validated ROC-AUC, established before held-out evaluation.

Model	CV AUC	Accuracy	Precision	Recall	F1	Test AUC
Logistic regression	0.911	0.803	0.767	0.821	0.793	0.932
Random forest	0.898	0.820	0.758	0.893	0.820	0.913

Logistic regression wins under the declared criterion. Random forest's higher test recall is reported rather than hidden; threshold choice should be tied to clinical costs in any future study.

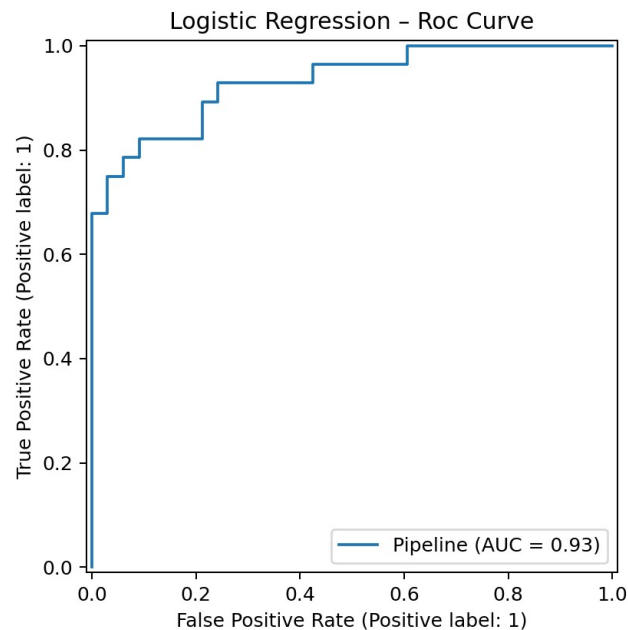


Figure 5. Held-out ROC curve for the selected pipeline.

4. Experiment tracking and reproducible packaging

Each MLflow run stores hyperparameters, five-fold CV ROC-AUC, five test metrics, confusion matrix, ROC curve, an input example and the complete sklearn pipeline. SQLite supports a reproducible local demonstration; MLFLOW_TRACKING_URI can point to a shared server.

The selected Joblib artifact contains preprocessing and classification together, preventing training-serving transformation skew. Metadata records the seed, timestamp, feature contract, sample counts, selection rule, metrics and limitations.

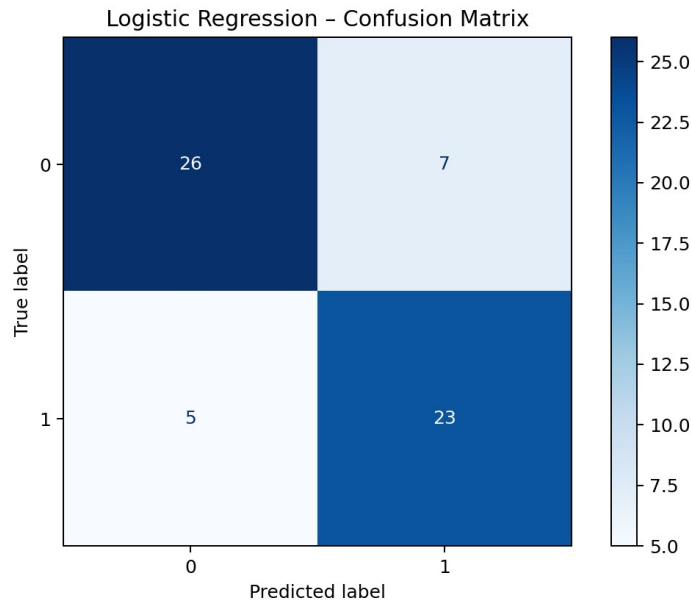


Figure 6. Selected-model confusion matrix on 61 held-out patients.

Required execution evidence

Insert your own screenshot of the MLflow experiment table and selected run here before submission. The screenshot must show both model runs, parameters, metrics and artifacts. Do not manufacture evidence.

5. API, testing and containerization

FastAPI exposes /health, /predict and /metrics. Pydantic validates all 13 fields, rejects unexpected keys and applies defensible range checks. Responses contain prediction, label, probability, model timestamp and a non-diagnostic disclaimer. Request logs omit supplied medical values.

Quality gates

- Data schema, target conversion, missingness and duplicate handling.
- Both candidate pipelines fit and return two-class probabilities.
- Health and prediction endpoints succeed with a controlled test artifact.
- Out-of-range patient inputs return HTTP 422.
- Verified result: 6 passed, 92% coverage; Ruff clean.

Container design

The Python 3.11 slim image runs as non-root UID 10 001, contains the trained model, exposes port 8000 and includes a health check. The build context excludes development data, tests, notebooks and MLflow state. Docker Compose adds Prometheus and Grafana for a one-command local demonstration.

Required execution evidence

Insert screenshots of passing tests, Docker build/run, Swagger request and returned prediction. Use the supplied sample_request.json and keep the disclaimer visible.

6. CI/CD and Kubernetes deployment

GitHub Actions creates a clean Python 3.11 environment, runs Ruff and Pytest, retrieves and validates the dataset, trains both candidates, uploads model/evaluation artifacts and builds the Docker image. Shell failures stop the job and the workflow uses read-only repository permission.

Kubernetes deploys two replicas behind a LoadBalancer Service. Readiness and liveness probes call /health. Explicit resource requests and limits control scheduling. Security settings require non-root execution, prohibit privilege escalation, drop Linux capabilities and mount a read-only root filesystem.

Local deployment sequence

- Start Minikube and point the Docker client at its image daemon.
- Build heart-risk-api:1.0.0, then apply the k8s directory.
- Wait for deployment rollout; confirm two Ready pods.
- Use minikube service heart-risk-api --url and call /predict.

Required execution evidence

Insert screenshots of a successful GitHub Actions run, uploaded artifacts, two ready Kubernetes pods, Service details and a prediction through the exposed endpoint.

7. Monitoring, logging and production response

Prometheus scrapes request counters, prediction counters and a latency histogram every five seconds. The provisioned Grafana dashboard displays request rate, p95 latency, outcomes by label and HTTP errors. Application logs record timestamp, method, route and status without patient feature payloads.

Proposed alert and response policy

- Availability: page on sustained health-check or 5xx failures; inspect logs and roll back.
- Latency: investigate p95 degradation against resource saturation and traffic changes.
- Prediction mix: review large sustained shifts as a possible input-distribution warning.
- Model quality: evaluate delayed, approved ground truth before any retraining decision.

Limitations and responsible use

The cohort is small, historical and geographically limited. Performance cannot establish external validity, calibration, fairness or clinical benefit. Binary sex coding is a dataset limitation. The 0.5 threshold ignores unequal error costs. Real use requires prospective multi-site validation, security/privacy review, subgroup evaluation, clinician governance and human override.

Required execution evidence

Generate several valid and invalid requests, then insert your Grafana dashboard screenshot showing traffic, latency, labels and errors.

8. Reproduction, conclusion and references

Clean reproduction

- Create a virtual environment and install requirements.txt.
- Run download_data.py, run_eda.py and train.py.
- Run pytest with coverage and ruff check.
- Start MLflow on port 5000 and FastAPI on port 8000.
- Build Docker/Compose; deploy k8s manifests with Minikube.

Submission links

GitHub repository: <https://github.com/amruthavallikuncha/mlops>

API or local access: [insert URL/command]

Demonstration video: [insert URL]

Conclusion

The project provides a consistent path from auditable raw data to a monitored deployment. Logistic regression is a defensible winner under the declared cross-validation criterion. The same fitted transformations are evaluated, packaged and served, while CI reproduces the evidence. The project demonstrates MLOps engineering rather than clinical readiness.

References

Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4X>

Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. IEEE Data Engineering Bulletin, 41(4), 39–45.

scikit-learn developers. Pipeline and composite estimators documentation.

FastAPI documentation. Request body and data validation.

Kubernetes documentation. Deployments, Services and probes.

Prometheus documentation. Metric and instrumentation practices.